

На рисунке 1 изображена диаграмма классов

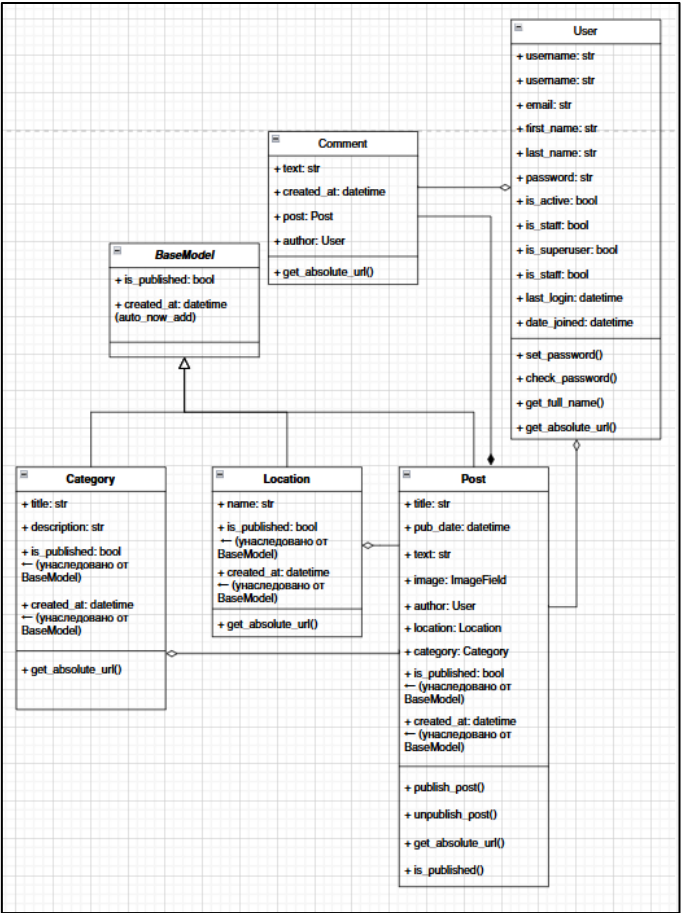


Рисунок 1 - Диаграмма классов

На рисунке 2 изображена диаграмма объектов.

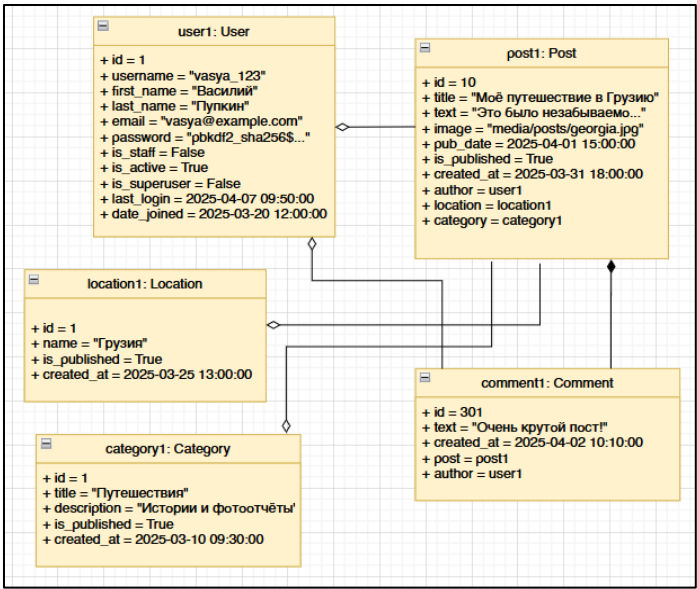


Рисунок 2 - Диаграмма объектов

На рисунках 3-6 представлены диаграммы в нотации IDEF0.

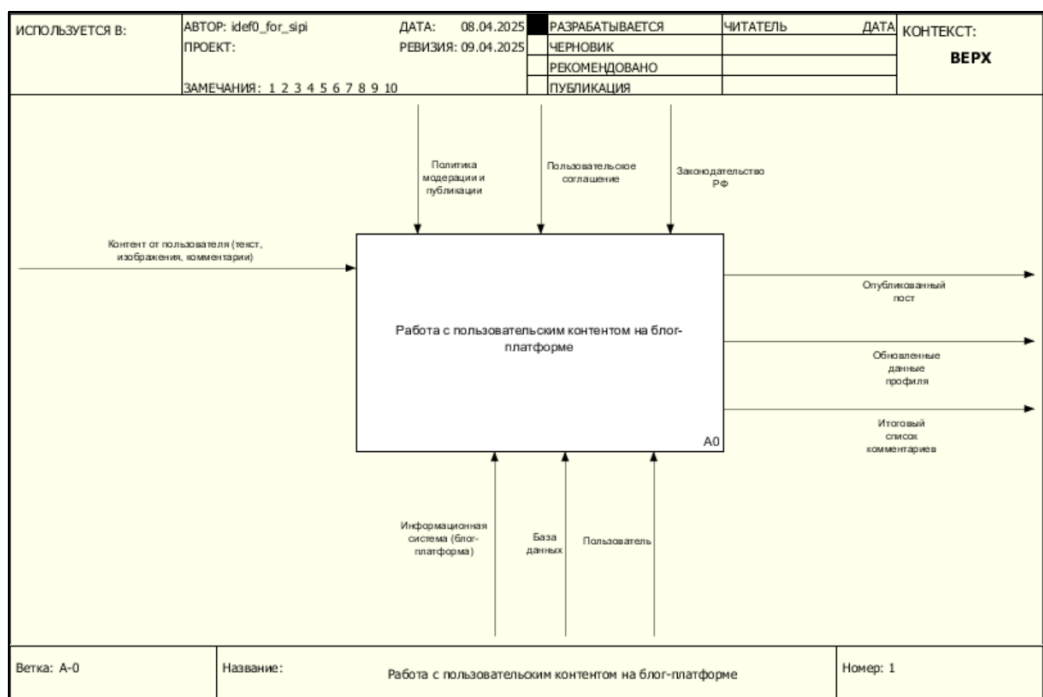


Рисунок 3 - Контекстная диаграмма в нотации IDEF0

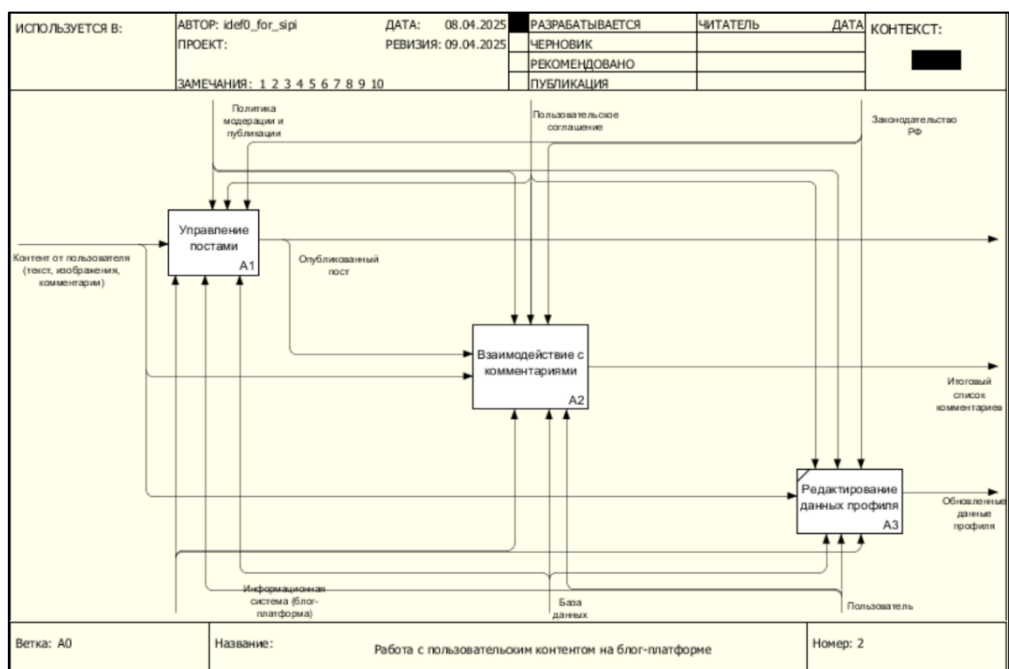


Рисунок 4 - Декомпозиция контекстной диаграммы

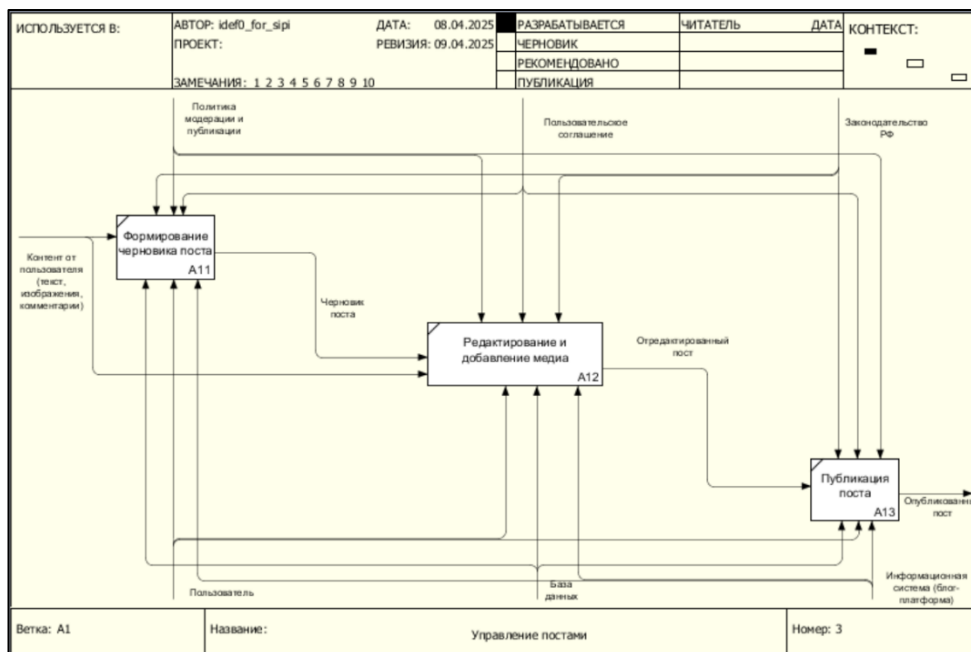


Рисунок 5 - Декомпозиция блока A1

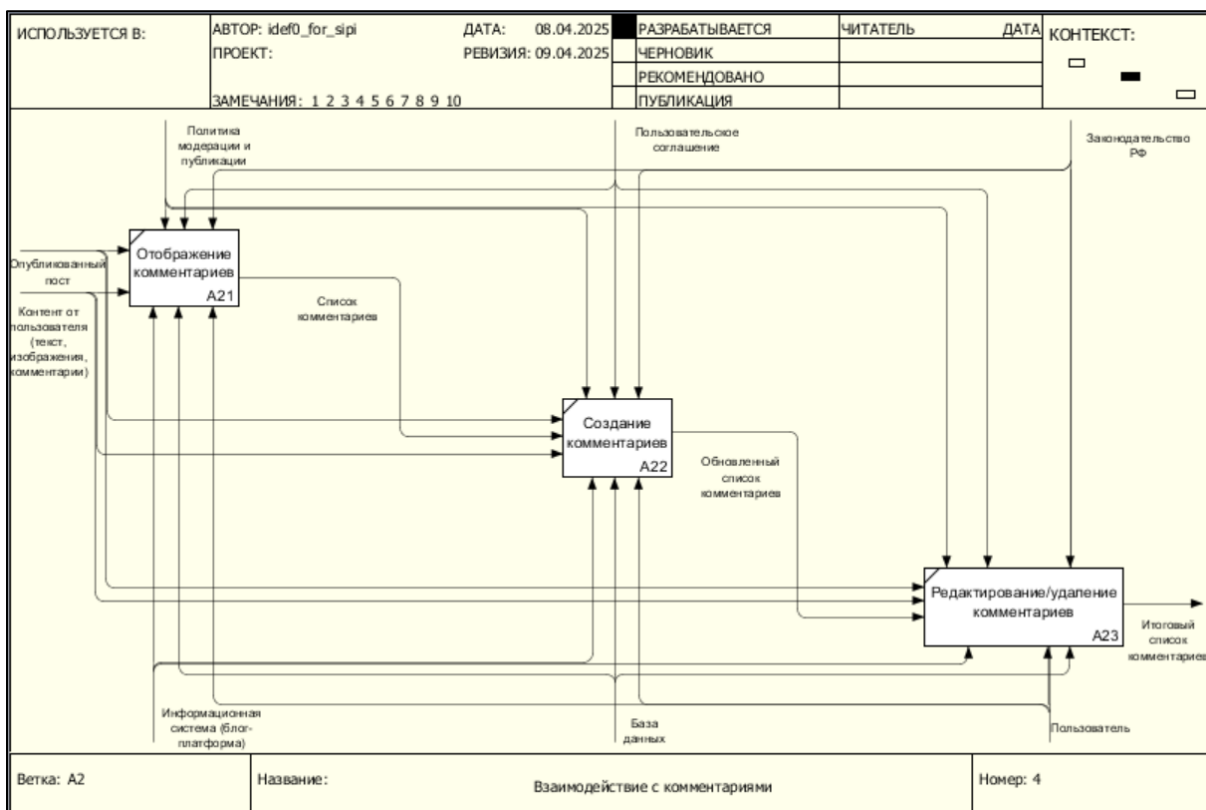


Рисунок 6 - Декомпозиция блока A2

На Рисунках 7-10 представлены диаграммы в нотации DFD.

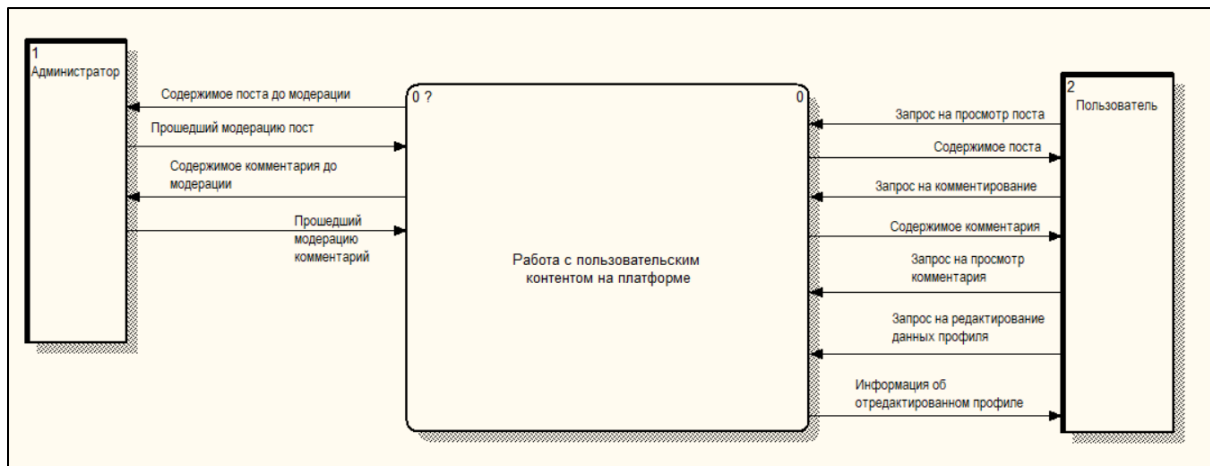


Рисунок 7 – Контекстная диаграмма в нотации DFD

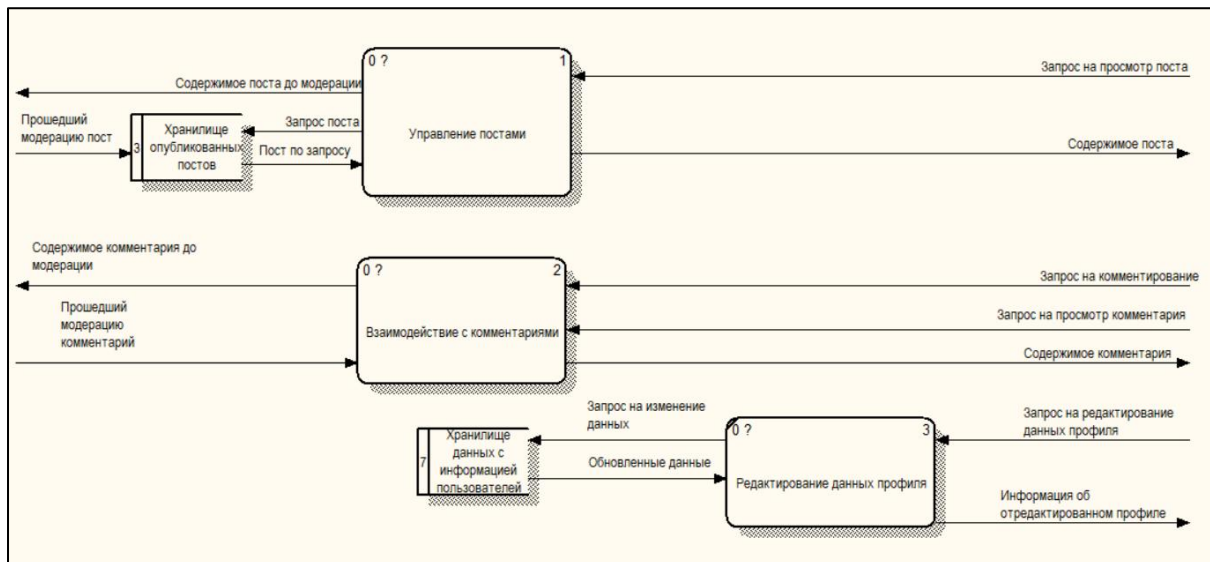


Рисунок 8 – Декомпозиция контекстной диаграммы

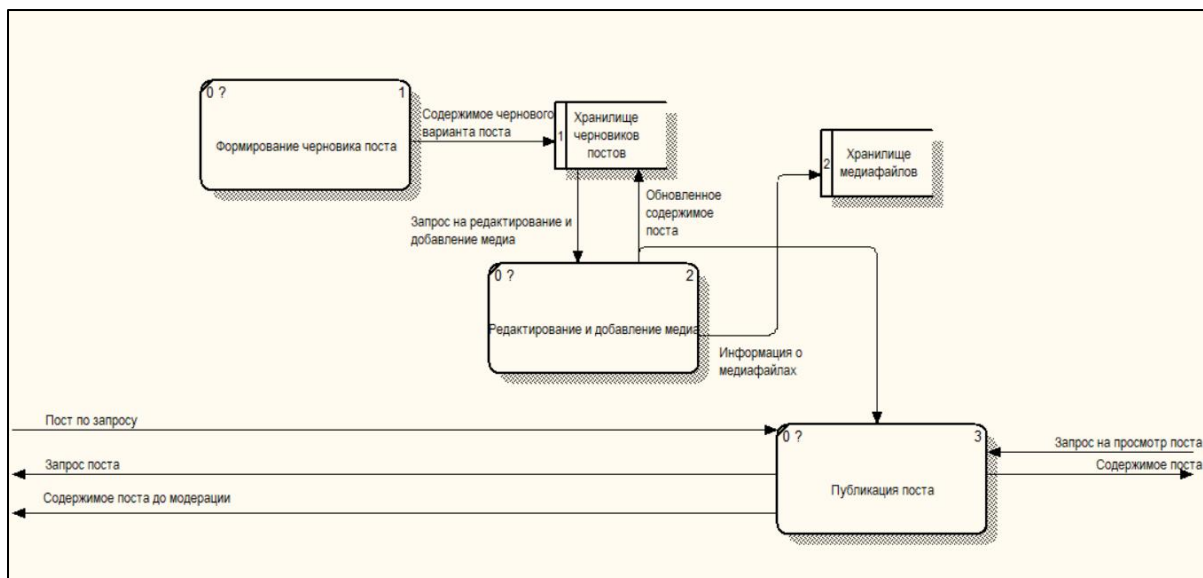


Рисунок 9 – Декомпозиция блока «Управление постами»

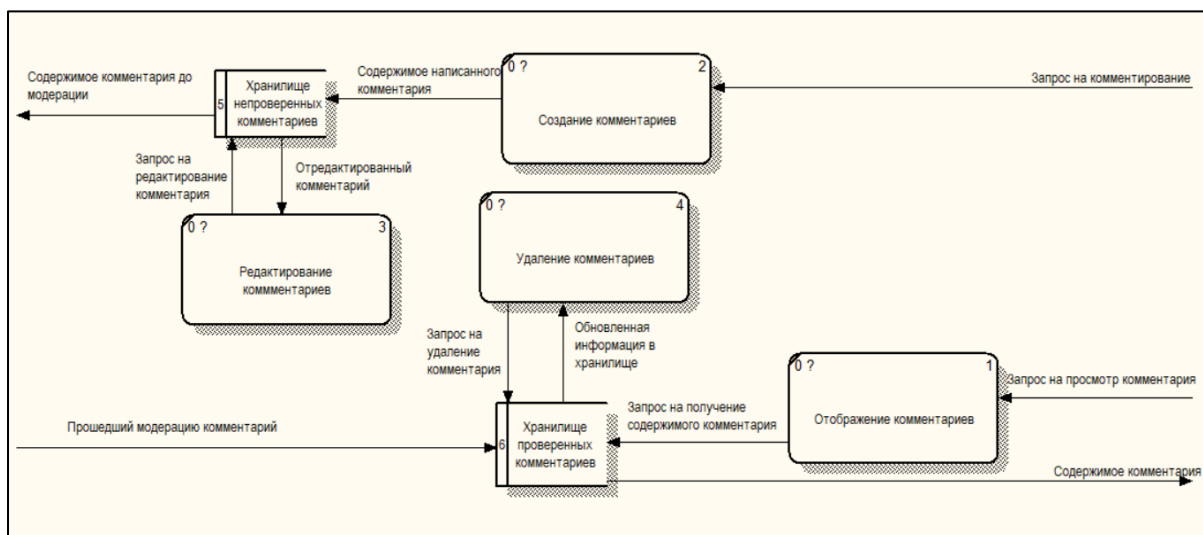


Рисунок 10 – Декомпозиция блока «Взаимодействие с комментариями»

В DFD диаграмме описывается процесс работы с пользовательским контентом на платформе.

Внешние сущности: администратор и пользователь.

Все действия пользователя на платформе (создание и редактирование постов, написание комментариев) проверяются администратором. Для того,

чтобы написать/просмотреть пост либо оставить/просмотреть комментарий необходимо отправлять запросы к хранилищам данных.

В процессе операций с постами изначально пользователь отправляет запрос на добавление своего поста в хранилище “Хранилище черновиков постов”, после чего черновой вариант поста передается далее для добавления медиафайлов, которые также поступают в “Хранилище медиафайлов”. Обновленное содержимое поста из хранилища “Хранилище черновиков постов” проходит через публикацию - отправляется администратору, после проверки администратор отправляет запрос на добавление информации о посте в “Хранилище опубликованных постов”.

В процессе операций с комментариями всё проходит аналогичным образом - пользователь отправляет запрос на добавление информации о комментарии в хранилище “Хранилище непроверенных комментариев”, а проверенный администратором комментарий публикуется и информация о нём передаётся в хранилище “Хранилище проверенных комментариев”.

6.3 Нормализованная логическая модель базы данных проекта

В базе данных есть следующие модели:

- модель поста;
- модель комментария;
- модель пользователя;
- модель категории (подборка постов определённой тематики);
- модель локации (указывается у опубликованного поста и отображает локацию, в которой выложен пост).

На Рисунке 11 представлена логическая база данных проекта.

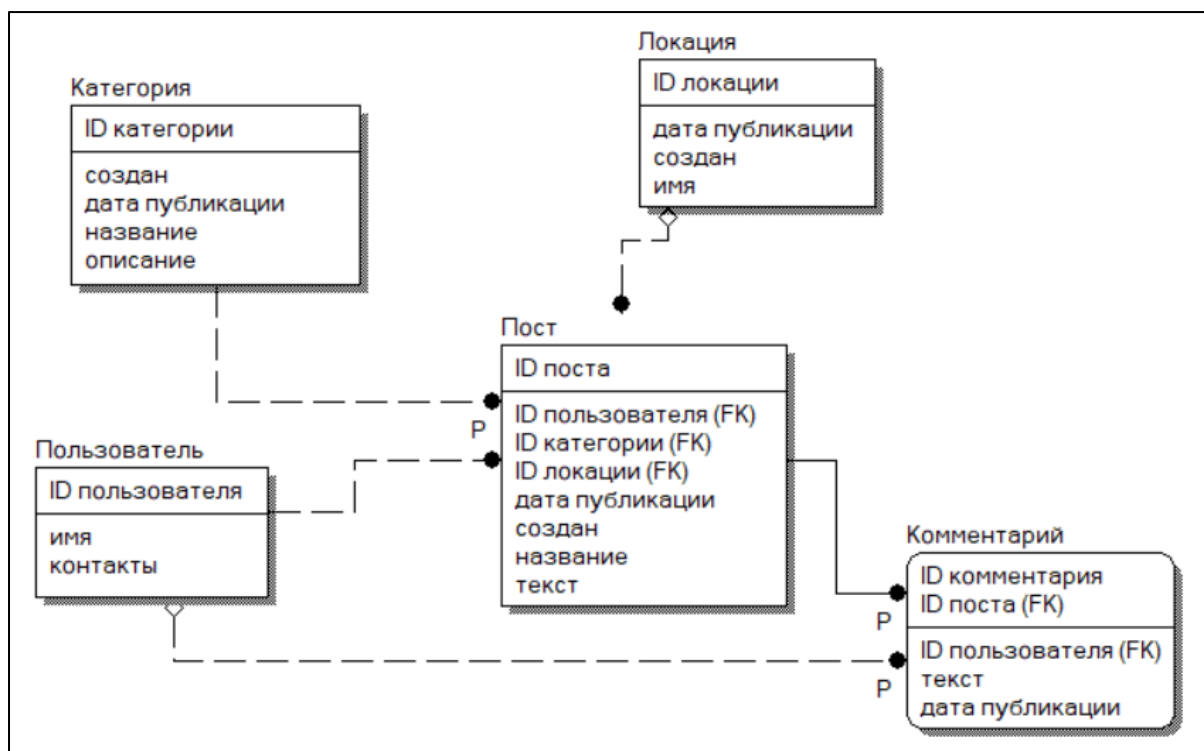


Рисунок 11 - Нормализованная логическая база данных проекта

В качестве архитектуры разрабатываемой системы была выбрана клиент-серверная архитектура. Выбор был сделан на основе преимуществ, предоставляемых данной архитектурой.

Клиент-серверная архитектура — это одна из самых распространенных архитектурных моделей для веб-приложений. Она предполагает разделение приложения на две основные части: клиент (мобильное приложение) и сервер (удаленный компьютер или облачный сервис).

Преимущества клиент-серверной архитектуры:

1. Централизованное управление данными: все данные и логика обработки запросов находятся на сервере, что упрощает управление данными, их обработку и защиту;
2. Масштабируемость: сервер можно масштабировать для обработки большего количества запросов без изменения клиентской части;
3. Безопасность: клиент не имеет прямого доступа к данным и бизнес-логике, что даёт возможность централизованно контролировать безопасность;

4. Облегчение тестирования: компоненты (API сервера, пользовательский интерфейс и др.) могут быть протестированы отдельно.
5. Упрощение обновления: сервер и клиент могут быть независимо обновлены или заменены.

Основные компоненты клиент-серверной архитектуры:

1. Клиент (Client):

Это веб-приложение, которое доступно на устройстве пользователя (компьютер, телефон).

Клиент отвечает за:

1. Отображение интерфейса (UI).
2. Взаимодействие с пользователем.
3. Отправку запросов на сервер и получение данных.

2. Сервер (Server):

Это удаленный компьютер или облачный сервис, который обрабатывает запросы от клиентов.

Сервер отвечает за:

1. Хранение данных (базы данных, файлы).
2. Обработку бизнес-логики (например, расчеты, проверка данных).
3. Управление безопасностью (аутентификация, авторизация).
4. Отправку ответов клиенту.

3. Сеть (Network):

Это канал связи между клиентом и сервером (обычно через интернет). Используются протоколы, такие как HTTP/HTTPS, WebSocket, TCP/IP.

Для реализации серверной части приложения будем использовать Platform as a service (PaaS). PaaS-платформа предоставит нам ресурсы для разворачивания своего сервера, который будет обрабатывать запросы пользователей.

В качестве PaaS была выбрана платформа PythonAnywhere.

PythonAnywhere — это облачная платформа для разработки, тестирования и хостинга Python-приложений. Она предоставляет удобное

онлайн-окружение, которое позволяет разработчикам работать с Python без необходимости настраивать сервер или локальное окружение. PythonAnywhere позволяет выполнять код, разрабатывать веб-приложения, запускать скрипты и выполнять другие задачи прямо в браузере.

Преимущества PythonAnywhere:

- Поддержка и интерфейс работы с БД: PythonAnywhere поддерживает несколько баз данных, включая SQLite, что позволяет использовать их для хранения данных веб-приложений.
- Скорость развертывания: Быстрое развертывание и тестирование веб-приложений.
- Малые расходы на инфраструктуру: PythonAnywhere берет на себя управление сервером, настройку и масштабирование, что значительно упрощает работу.
- Поддержка Python: Платформа ориентирована на работу с Python, что делает ее идеальной для разработчиков Python-приложений.

Основным инструментом разработки выберем язык программирования Python.

Python — это высокоуровневый, интерпретируемый язык программирования. Он поддерживает несколько парадигм программирования, таких как объектно-ориентированное, функциональное и процедурное программирование.

Основные преимущества Python:

1. Простота и читаемость: Легкий синтаксис, который делает код понятным и легко поддерживаемым.
2. Быстрая разработка: Благодаря высокой абстракции и большому количеству готовых библиотек код можно писать быстрее.
3. Кросс-платформенность: Работает на различных операционных системах, таких как Windows, Linux, macOS.
4. Большое сообщество: Активное сообщество, большое количество документации, обучающих материалов и библиотек.
5. Множество библиотек и фреймворков: Для разных задач (например, Django для веб-разработки, Pandas для анализа данных, TensorFlow для машинного обучения).
6. Масштабируемость: Python подходит как для малых, так и для крупных проектов.

7. Поддержка разных парадигм программирования: ООП, функциональное, процедурное программирование.
8. Подходит для разных областей: Научные вычисления, машинное обучение, веб-разработка, автоматизация, анализ данных.

Для осуществления серверной логики приложения используем фреймворк Django.

Django — это высокоуровневый фреймворк для разработки веб-приложений на языке Python. Он предназначен для упрощения и ускорения создания сложных, масштабируемых веб-сайтов и сервисов.

Преимущества Django:

1. Быстрая разработка: Django позволяет быстро создавать и развертывать веб-приложения благодаря набору встроенных инструментов и автоматизации.
2. Чистота кода: Фреймворк поощряет использование принципов DRY (Don't Repeat Yourself), что способствует написанию лаконичного и поддерживаемого кода.
3. Масштабируемость: Подходит для создания как маленьких сайтов, так и крупных масштабируемых приложений.
4. Безопасность: Django включает множество встроенных средств для защиты приложений, например, от атак SQL-инъекций, CSRF, XSS и других.
5. Активное сообщество: Огромное количество документации, плагинов и поддержка сообщества.

Основные особенности Django:

1. Полный стек: Django предоставляет готовый набор инструментов для решения большинства задач, необходимых для создания веб-приложений (база данных, маршрутизация, аутентификация, формы и т.д.).
2. ORM (Object-Relational Mapping): Django включает мощный механизм ORM, который позволяет работать с базами данных через Python-классы, избегая необходимости писать SQL-запросы вручную. Это упрощает взаимодействие с базой данных и ускоряет разработку.
3. Шаблоны: Django использует систему шаблонов для генерации динамического HTML. Это облегчает разделение логики и

представления, что способствует чистоте кода и лучшей поддерживаемости.

4. Модульность: В Django можно легко подключать сторонние пакеты и модули для добавления новых функций, таких как аутентификация через социальные сети, отправка email, кеширование и т.д.
5. Система маршрутизации: Django предоставляет гибкую систему маршрутизации URL, которая позволяет связать URL-пути с соответствующими обработчиками.
6. Тестирование: Django поддерживает модульное тестирование из коробки, что позволяет автоматизировать тестирование приложений и предотвращать ошибки в коде.

В качестве базы данных выберем SQLite.

SQLite — это легковесная, встраиваемая база данных, которая использует обычные файлы для хранения данных, а не серверное приложение. Она часто используется в мобильных приложениях, встроенных системах и небольших веб-приложениях. SQLite является сервером без серверной части, что делает её простым и удобным инструментом для работы с базами данных.

Преимущества SQLite:

1. Легковесность: SQLite не требует установки или настройки отдельного серверного приложения, что делает её компактной и быстрой.
2. Производительность: Быстрое выполнение операций, особенно для небольших приложений и проектов.
3. Простота использования: Использование SQLite не требует сложной конфигурации и управления сервером баз данных.
4. Кросс-платформенность: Работает на всех основных операционных системах (Windows, Linux, macOS).
5. Поддержка SQL: Поддерживает стандартный язык SQL для работы с базами данных.
6. Подходит для небольших и средних проектов: Отлично подходит для использования в мобильных приложениях, веб-сайтах с малым трафиком, и для тестирования.

7. Низкие требования к ресурсам: SQLite не требует много памяти или процессорных ресурсов, что делает её идеальной для ограниченных устройств и приложений.

Для оформления клиентской части приложения используем фреймворк Bootstrap.

Bootstrap — это популярный фреймворк для фронтенд-разработки, который предоставляет набор готовых стилей, компонентов и инструментов для создания адаптивных и современных веб-интерфейсов. Он включает CSS и JavaScript библиотеки для быстрого создания сеток, кнопок, форм, навигации и других элементов.

Преимущества Bootstrap:

1. Быстрая разработка: Использование готовых компонентов и стилей ускоряет процесс создания интерфейсов.
2. Адаптивный дизайн: Встроенная поддержка мобильных устройств и адаптивных сеток, что упрощает создание сайтов, хорошо отображающихся на разных устройствах.
3. Гибкость и настройка: Легко кастомизируется с помощью переменных и пользовательских стилей.
4. Широкое сообщество: Большое количество документации, примеров и активное сообщество разработчиков.
5. Кросс-браузерная совместимость: Работает во всех современных браузерах.
6. Готовые компоненты: Включает множество готовых элементов UI, таких как кнопки, формы, карточки, модальные окна, что упрощает дизайн.

Для работы с изображениями (загрузка, сохранение, обработка) используем библиотеку Pillow.

Pillow — это библиотека Python для обработки изображений. Она предоставляет простой интерфейс для работы с изображениями, включая открытие, изменение, сохранение и преобразование изображений различных форматов.

Преимущества Pillow:

1. Широкий поддерживаемый формат изображений: Pillow поддерживает множество форматов изображений, включая JPEG, PNG, BMP, GIF, TIFF и другие.
2. Простота в использовании: Библиотека имеет простой и интуитивно понятный интерфейс для обработки изображений.
3. Мощные инструменты для редактирования: Поддержка различных операций, таких как обрезка, изменение размера, поворот, изменение яркости и контрастности, фильтрация и наложение текста.
4. Работа с альфа-каналом: Поддержка прозрачности в изображениях (например, в формате PNG).
5. Поддержка графики: Позволяет создавать и обрабатывать графику, например, рисовать текст, линии, прямоугольники, окружности и другие фигуры на изображениях.

Архитектурная диаграмма клиент-серверной архитектуры показана на рисунке 7.1.

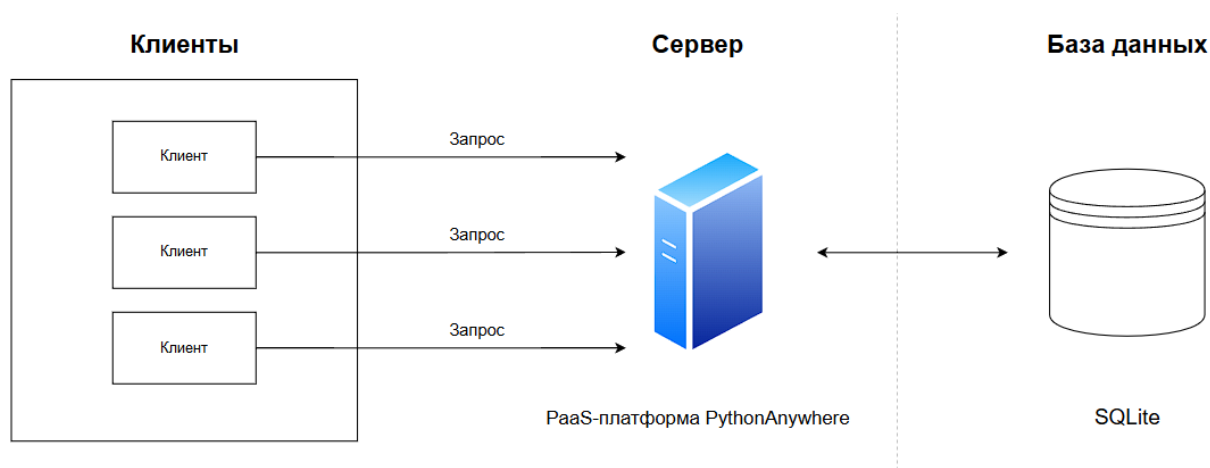


Рисунок 7.1 – Диаграмма клиент-серверной архитектуры

Таблица 7.1 – Матрица требований

№	Требование	Суть	Критерий проверки	Компонент архитектуры
---	------------	------	-------------------	-----------------------

1	Веб-интерфейс			
1.1	Регистрация пользователя	“Должна быть реализована функция регистрации нового пользователя”	Новый пользователь зарегистрирован	Клиентская часть Серверная часть
1.2	Авторизация пользователя	“Должна быть реализована функция авторизации зарегистрированного пользователя”	Зарегистрированный пользователь вошел в аккаунт в приложении	Клиентская часть Серверная часть
1.3	Просмотр постов	“Должна быть реализована возможность просмотра постов других пользователей, отсортированных по дате публикации”	На главной странице отображаются посты других пользователей в хронологическом порядке	Клиентская часть
1.4	Фильтрация поиска постов	“Должна быть реализована возможность фильтровать посты по категориям”	При фильтрации постов по категориям все посты отражают корректные запрошенные данные	Клиентская часть Серверная часть

1.5	Создание собственного поста	“Должна быть реализована возможность написать и опубликовать свой пост с добавлением категорий и изображений”	Новый пост пользователя есть в базе данных, опубликован и виден остальным пользователям	Клиентская часть Серверная часть
1.6	Редактирование и удаление своих постов	“Должна быть реализована возможность редактировать или удалить свой опубликованный пост”	Пост пользователя изменился или стал недоступен для других пользователей	Клиентская часть Серверная часть
1.7	Скрытие своих постов	“Должна быть реализована возможность скрыть собственный пост от отображения другим пользователям”	Пост пользователя не выводится при поиске и на странице всех его постов	Клиентская часть
1.8	Отображение всех постов пользователя	“Должна быть реализована возможность просмотра всех постов конкретного пользователя”	Доступна страница со всеми постами данного пользователя	Клиентская часть

1.9	Просмотр комментариев к посту	“Должна быть реализована возможность отображения комментариев под постами для всех пользователей”	На странице поста отображаются комментарии	Клиентская часть Серверная часть
1.10	Написание комментариев к посту	“Должна быть реализована возможность написать и опубликовать комментарий к посту для зарегистрированных пользователей”	Комментарий добавлен в базу данных и отображается под постом	Клиентская часть Серверная часть
1.11	Редактирование профиля	“Должна быть реализована возможность заполнить и редактировать свой профиль”	При заполнении формы редактирования профиля новые данные сохраняются в базе данных	Клиентская часть Серверная часть
2	Нефункциональные требования			
2.1	Время отклика	“Время отклика интерфейса не более 400 мс.”	Все операции, изображенные на диаграмме последовательности, занимают не более 400 мс.	Серверная часть

2.2	Нагрузка интерфейса	“Когнитивная нагрузка интерфейса ниже 7 единиц”	На основных экранах не более 7 ключевых элементов	Клиентская часть
2.3	Кроссбраузерность	“Веб-приложение должно быть кроссбраузерным и одинаково функционировать во всех популярных браузерах”	Приложение прошло тесты на всех указанных браузерах, и функциональность идентична	Клиентская часть
2.4	Резервное копирование	“Полное резервное копирование базы данных клиентов должно производиться не менее чем 1 раз в неделю”	Наличие резервных копий за последнюю неделю	Серверная часть
2.5	Snapshot базы данных	“Snapshot базы данных должен производиться не менее чем 1 раз в сутки”	Наличие snapshot за последние 24 часа	Серверная часть
2.6	Кэширование	“Кэширование часто запрашиваемых данных для уменьшения времени отклика”	Уменьшение времени отклика до и после внедрения кэширования	Серверная часть

2.7	Устойчивость при нагрузке	“При нагрузке в 200 запросов в секунду система должна выдавать пользователю ответ на запрос в пределах 3 с”	Нагрузочное тестирование системы с 200 запросами в секунду успешно	Серверная часть
2.8	Надежность	“Приложение должно иметь время uptime в год 95%”	Среднее время простоя приложения 18 дней и 6 часов	Серверная часть
2.9	Восстановление	“Время восстановления системы после сбоя не должно превышать 6 часов”	При проведении тестового сбоя время восстановления не более 6 часов	Серверная часть
2.10	Безопасность	“Для хэширования пароля необходимо использовать алгоритм SHA256”	Все пароли захэшированы по алгоритму SHA256	Серверная часть
2.11	Защита от атак	“Система должна быть защищена от распространенных атак, таких как SQL-инъекции, XSS и CSRF”	Тесты на уязвимость к указанным атакам пройдены	Серверная часть

2.1 2	Защита при авторизации	“Авторизация пользователей должна быть построена на использовании JWT”	Работоспособность авторизации пользователей с помощью JWT-токенов проверена	Клиентская часть Серверная часть
----------	------------------------	--	---	---

Для разработки технического задания был выбран ГОСТ 34.602-2020

Преимущества ГОСТ 34.602-2020:

- Современность:

ГОСТ 34.602-202 был разработан с учетом современных требований к разработке программного обеспечения и информационных систем, что делает его более актуальным в условиях быстро меняющихся технологий.

- Улучшенная структура:

Новый стандарт имеет более четкую и логически структурированную организацию, что облегчает понимание и применение требований.

- Фокус на жизненном цикле:

ГОСТ 34.602-202 акцентирует внимание на всех этапах жизненного цикла программного обеспечения, включая эксплуатацию и сопровождение, что способствует более полному управлению проектами.

- Интеграция с другими стандартами:

ГОСТ 34.602-202 интегрируется с другими международными стандартами, что упрощает процесс сертификации и соответствия.

8.2. Техническое задание по ГОСТу.

Техническое задание:

1. Введение

1.1. Наименование проекта: Веб-приложение "Uniqueblog".

1.2. Цель разработки: Создание веб-приложения, которое позволит пользователям публиковать посты, чтобы делиться важной для них информацией.

2. Общие сведения

2.1. Заказчик: РТУ МИРЭА

2.2. Исполнитель: Студенты группы ИКБО-30-22

2.3. Сроки выполнения: до 31.05.2025

3. Описание функциональности

3.1 Регистрация пользователя

3.2 Авторизация пользователя

3.3 Просмотр постов

3.4 Фильтрация постов по категориям

3.5 Создание собственного поста

3.6 Редактирование и удаление своих постов

3.7 Скрытие своих постов

3.8 Просмотр комментариев к посту

3.9 Написание комментариев к посту

3.10 Редактирование профиля

4. Требования к интерфейсу

4.1. Дизайн:

4.1.1 Приложение должно иметь современный интерфейс.

4.2. Удобство использования:

4.2.1 Навигация по приложению должна быть простой и логичной, чтобы пользователи могли быстро находить нужные функции.

5. Нефункциональные требования

5.1. Производительность:

5.1.1 Время отклика интерфейса не более 400 мс

5.1.2 Когнитивная нагрузка интерфейса ниже 7 единиц

5.1.3 При нагрузке в 200 запросов в секунду система должна выдавать пользователю ответ на запрос в пределах 3 с

5.2. Безопасность:

5.2.1. Все данные пользователей должны передаваться по защищенному протоколу HTTPS

5.2.2. Приложение должно использовать шифрование для хранения персональных данных пользователя

5.2.3. Доступ к данным аккаунта пользователя осуществляется через процесс входа в аккаунт; пароль должен запрашиваться при каждом повторном входе

5.2.4 Система должна быть защищена от распространенных атак, таких как SQL-инъекции, XSS и CSRF

5.3. Совместимость:

5.3.1. Веб-приложение должно быть кроссбраузерным

5.3.2. Поддержка различных языков.

5.4. Тестирование:

5.4.1. Проведение функционального тестирования, тестирования производительности и безопасности.

6. Требования к системе

6.1. Требования к надежности

6.1.1. Приложение должно обеспечивать непрерывную работу не менее 95% времени в месяц.

6.1.2. Время восстановления после сбоя не должно превышать 6 часов.

6.1.3. Все критические ошибки (краши, зависания) должны регистрироваться в журнале с возможностью их последующего анализа.

6.2. Требования к сохранности данных при авариях

- 6.2.1. Данные о постах и блогах пользователей должны храниться в защищенной базе данных и сохраняться при аварийном завершении работы.
- 6.2.2. При потере соединения с сервером приложение должно обеспечивать локальное кеширование данных с последующей их синхронизацией.
- 6.2.3 Полное резервное копирование базы данных пользователей должно производиться не менее чем 1 раз в неделю
- 6.2.4. Ошибки некритичных модулей (например, загрузка изображений) не должны приводить к полному отказу системы.
- 6.2.5. При возникновении критической ошибки веб-приложение должно корректно завершать работу, предупреждая пользователя.
- 6.3. Требования к восстановлению после отказов
 - 6.3.1. При перезапуске приложения все незавершенные операции (написание поста, авторизация, комментирование поста) должны быть восстановлены.
 - 6.3.2. В случае сбоя сетевого соединения приложение должно автоматически переподключаться и синхронизировать данные.
- 6.5. Требование к безопасности
 - 6.5.1. Доступ к различным функциям веб-приложения должен быть разграничен на основе ролей (например, пользователь, администратор).

6.5.2. Веб-приложение не должно запрашивать и хранить избыточные персональные данные, не связанные с основной функциональностью

7. Требования к документации

7.1. Документация для пользователя:

7.1.1. Руководство пользователя с описанием функциональности приложения.

7.2. Техническая документация:

7.2.1. Техническое описание архитектуры приложения, используемых технологий и API.

8. Этапы разработки

8.1. Этапы:

8.1.1. Анализ требований. (Аналитик, срок до 05.03.2025)

8.1.2. Проектирование интерфейса. (Разработчики, срок до 01.04.2025)

8.1.3. Разработка приложения. (Разработчики, срок до 19.04.2025)

8.1.4. Тестирование. (Тестировщик, срок до 01.05.2025)

8.1.5. Внедрение и поддержка. (Менеджер проекта, срок до 31.05.2025)